

IFPI CACAM - Banco de Dados I

Prof. Nairon Viana (nairon.viana@ifpi.edu.br)

Curso: Tecnólogo em ADS - 2026/02 (Módulo II)



Informações da Disciplina



Visão Geral

- **Carga horária:** 60h
- **Ementa:** Introdução a Banco de Dados. Arquiteturas de SGBDs. Modelos de Dados. Modelo Entidade-Relacionamento. Modelo Relacional. Dependência Funcional e Normalização de Dados. Álgebra Relacional. Introdução à Structured Query Language (Data Definition Language - DDL e Data Manipulation Language - DML).
- **Objetivo:** Desenvolver os principais conceitos e aspectos operacionais de bancos de dados. Entender os diversos modelos de dados existentes. Projetar e construir bancos de dados.

Competências e Habilidades

1

Fundamentos

Compreender conceitos fundamentais de bancos de dados

2

Modelagem

Modelar dados usando diagramas entidade-relacionamento

3

SGDB

Implementar bancos de dados relacionais com SQL em um SGDB

4

Otimização

Aplicar normalização e otimização de estruturas

Metodologia das Aulas

Aulas Teóricas

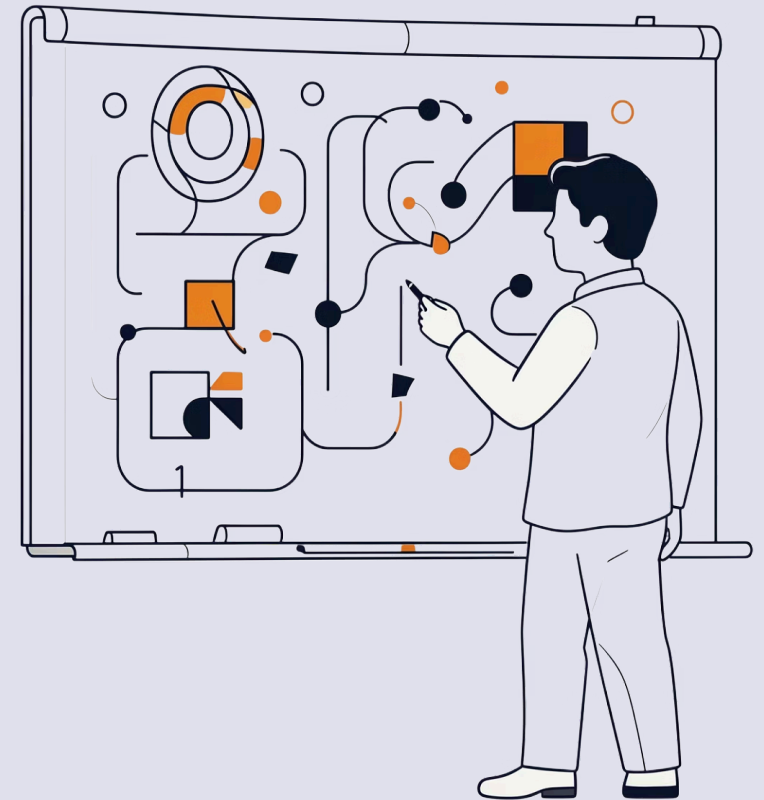
Conceitos, modelos e arquitetura de sistemas de banco de dados

Aulas Práticas

Modelagem e SQL em ambiente de laboratório supervisionado

Projeto Integrador

Desenvolvimento de um banco de dados completo e funcional



Avaliações

1 Provas Teóricas

Avaliação de conceitos e modelagem de dados

3 Projeto Final

Entrega de banco de dados funcional completo

2 Exercícios Práticos

Atividades de SQL e normalização em laboratório



Bibliografia Recomendada(ver plano)

Elmasri & Navathe

Sistemas de Banco de Dados — Referência principal da disciplina, cobrindo fundamentos, modelagem relacional e SQL de forma aprofundada.

Date, C.J.

Introdução a Sistemas de Banco de Dados — Obra clássica com rigor teórico sobre o modelo relacional e álgebra relacional.

Documentação Oficial

Documentação dos SGBDs **PostgreSQL** e **MySQL** — Leitura prática indispensável para as atividades de laboratório.

O que é um Banco de Dados?

Segundo **Elmasri & Navathe**, um banco de dados é uma **coleção de dados relacionados**, organizada para atender a **múltiplas aplicações**.

Miniworld

Representa algum aspecto do **mundo real** — ou seja, um recorte da realidade que queremos modelar.

Coerência lógica

É uma coleção de dados **logicamente coerente**, com significado próprio e relacionamento entre os elementos.

Propósito específico

É **projetado, construído e alimentado** para atender a um objetivo definido, como registrar, consultar ou controlar informações.

Para que serve?

Facilita a **recuperação**, a **atualização** e a **manutenção** dos dados com eficiência e organização.

Por que não usar apenas arquivos?

Sistemas de arquivos tradicionais tendem a gerar **redundância**, **inconsistência** e dificuldade para compartilhar dados entre aplicações.

Em resumo: um banco de dados organiza dados de forma integrada para representar a realidade, apoiar aplicações e permitir acesso confiável e eficiente.

Conceitos Fundamentais de Banco de Dados

Banco de Dados

Representa o arquivo físico de dados, armazenado em dispositivos periféricos, onde estão armazenados os dados de diversos sistemas, para consulta e atualização pelo usuário.

Dado

É o valor do campo quando é armazenado no Banco de Dados.

Tabelas Lógicas

Representam as estruturas de armazenamento de dados (arquivos) dos sistemas.

S.G.D.B.

Sistema Gerenciador de Banco de Dados: é o software responsável pelo gerenciamento (armazenamento e recuperação) dos dados no Banco de Dados.

Conteúdo do campo

É o valor do campo armazenado no Banco de Dados.

Modelo de Banco de Dados

Modelo Relacional, Modelo Hierárquico e Modelo em Rede. Representa a estrutura física no qual o armazenamento dos dados foram projetados. O modelo identifica a estrutura interna de recuperação e armazenamento dos dados no qual o SGBD foi projetado.

Propriedades Genéricas de um Banco de Dados

1

Coleção de dados relacionados

Um BD é uma coleção de dados logicamente relacionados, com algum significado.

Implica em que associações aleatórias de dados não possam ser chamadas de bases de dados.

2

Propósito específico

Um BD é projetado, construído e preenchido com dados para um propósito específico.

Possui um grupo de usuários e algumas aplicações concebidas para atender a esses usuários.

3

Representação do mundo real

Um BD é a representação de algum aspecto do mundo real.

Nos leva a que, qualquer mudança na realidade, provocará mudanças nessa base de dados.

Visões de um Banco de Dados

Visão Interna

É aquela vista pelo responsável pela manutenção e desenvolvimento do SGBD.

Existe a preocupação com a forma de recuperação e manipulação dos dados dentro do Banco de Dados.

Visão Conceitual

É aquela vista pelo analista de desenvolvimento e pelo administrador das bases de dados.

Existe a preocupação na definição de normas e procedimentos para manipulação dos dados, para garantir a sua segurança e confiabilidade.

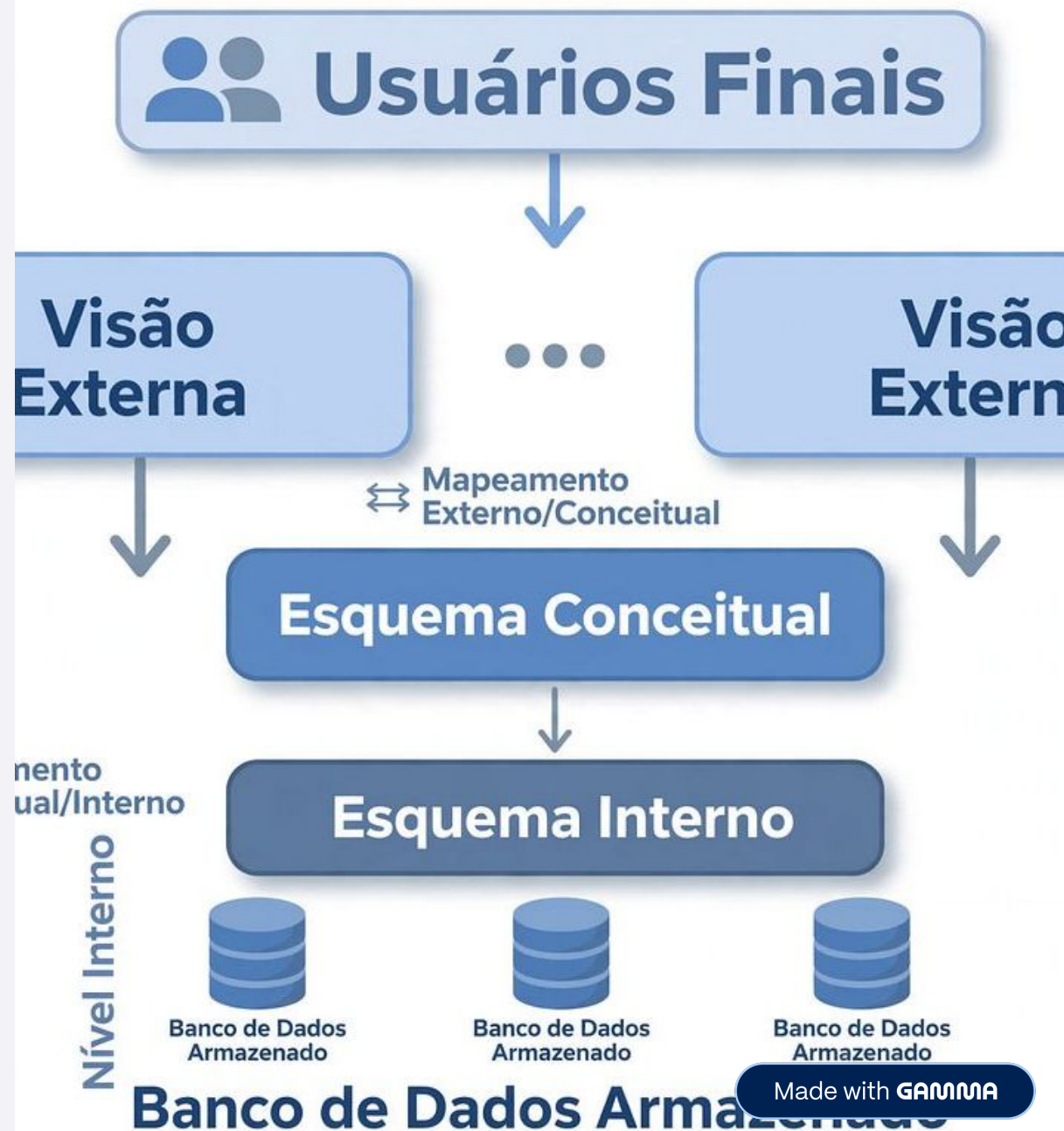
Desenvolvimento de sistemas e programas aplicativos e a definição no banco de dados de novos arquivos e campos.

Visão Externa

É aquela vista pelo usuário que opera os sistemas aplicativos, através de interfaces desenvolvidas pelo analista (programas).

Busca o atendimento de suas necessidades.

Visões de um BD





CAPÍTULO

Histórico dos Bancos de Dados

Origens e evolução dos sistemas de gerenciamento de dados.

Antes dos SGBDs: Arquivos

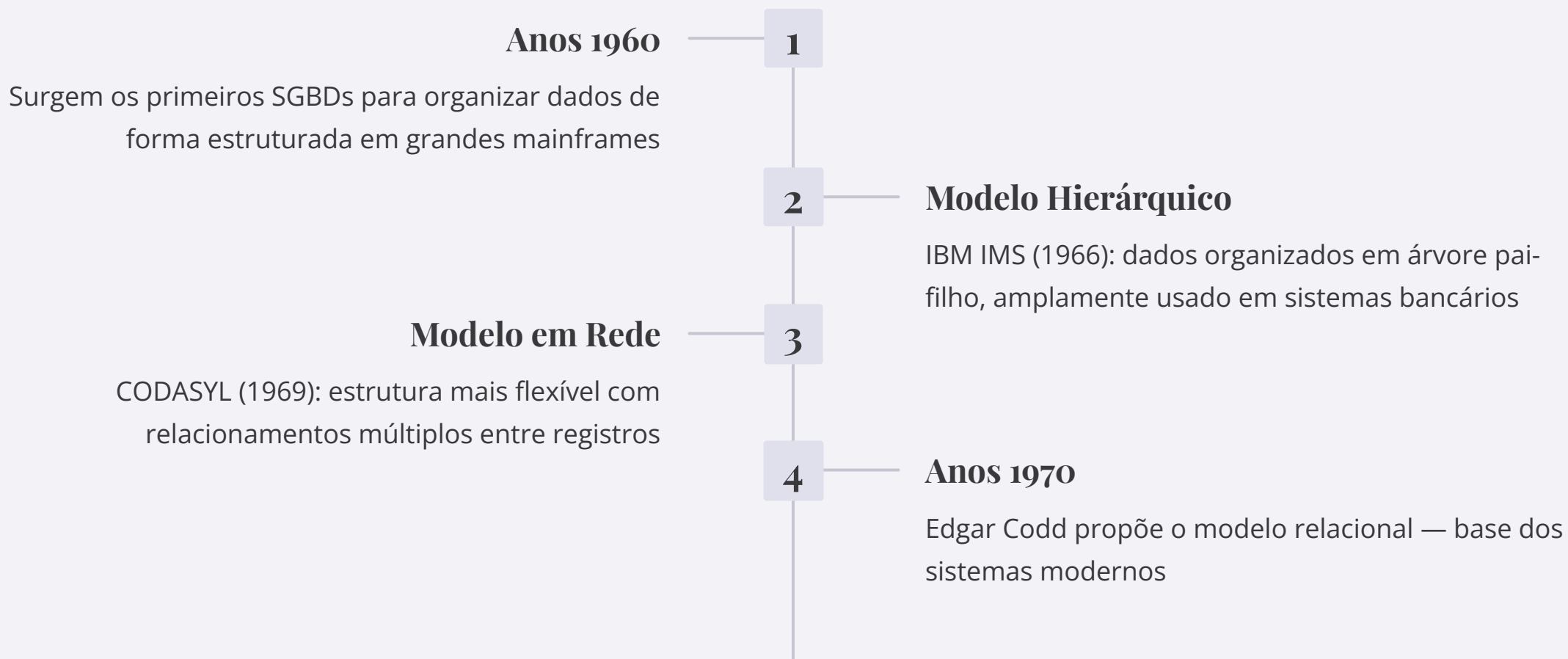
O problema dos arquivos simples

Antes dos sistemas de banco de dados, organizações armazenavam todas as suas informações em arquivos de texto ou binários isolados — uma abordagem que rapidamente revelou graves limitações.

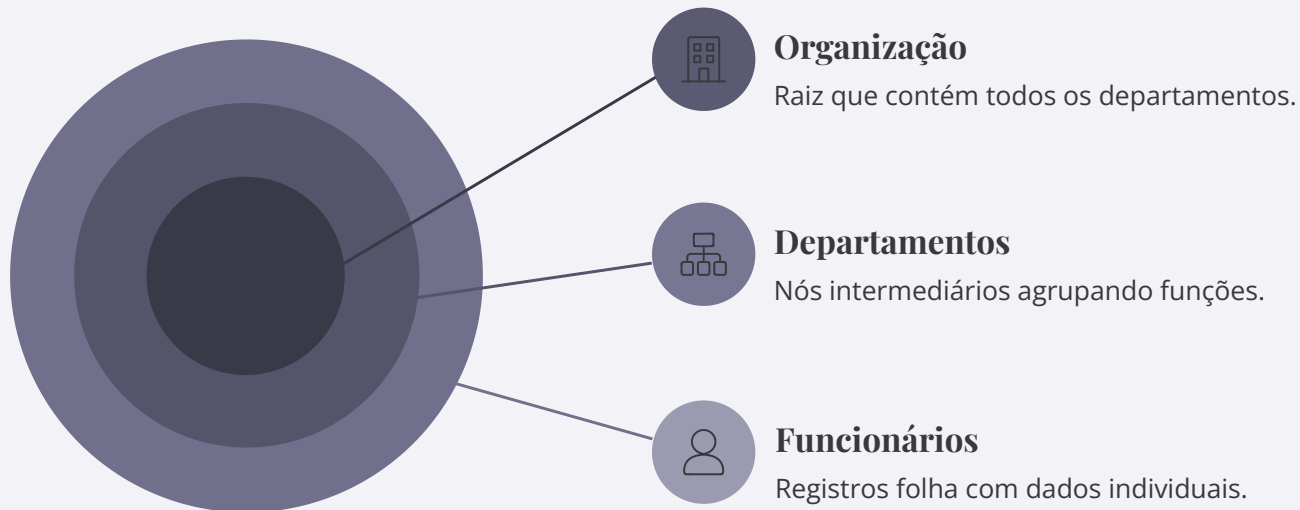
- Dados armazenados em arquivos simples e isolados
- **Redundância e inconsistência:** o mesmo dado replicado em múltiplos arquivos
- Compartilhamento entre sistemas era difícil e propenso a erros
- Ausência de controle de concorrência ou segurança de acesso



Primeiros Sistemas de Gerenciamento



Modelo Hierárquico



Estrutura em árvore (pai-filho)

No modelo hierárquico, cada registro possui exatamente um pai, formando uma estrutura de árvore rígida — semelhante ao sistema de arquivos de um sistema operacional.

- Dados organizados em hierarquia de pai para filho
- Exemplo clássico: sistema de arquivos do SO
- **Limitação:** difícil representar relacionamentos complexos ou muitos-para-muitos

📌 Este modelo foi a base para entendermos **por que** o modelo relacional representou uma revolução.

Modelo em Rede

O modelo em rede surgiu como uma evolução do modelo hierárquico, permitindo relações de dados mais flexíveis e conectadas.

Características principais

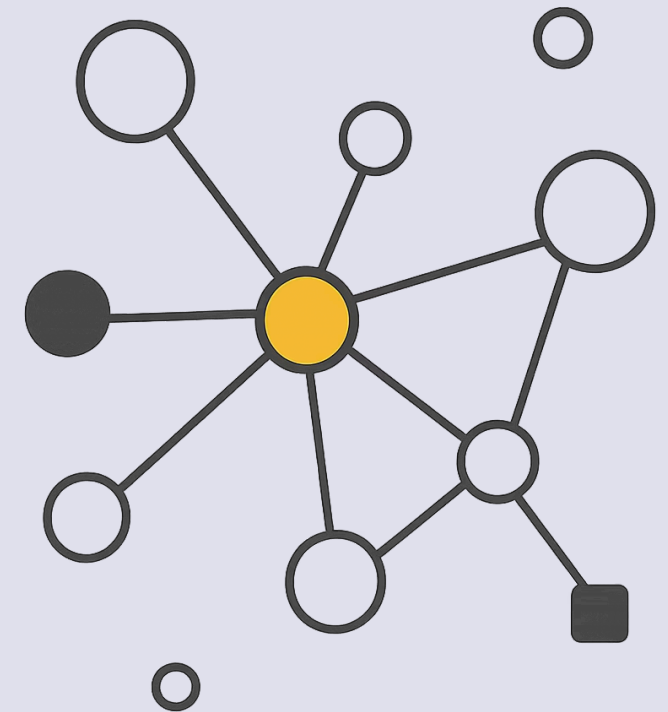
- Permite múltiplos relacionamentos entre registros, não apenas pai-filho
- Estrutura de grafo: registros podem ter múltiplos pais e múltiplos filhos
- Mais flexível que o modelo hierárquico
- Baseado no padrão CODASYL (1969)

Vantagens

- Representa relacionamentos complexos e muitos-para-muitos
- Melhor que o modelo hierárquico para dados com múltiplas relações

Desvantagens

- Complexo de implementar
- Difícil de consultar, pois requer navegação manual pelos relacionamentos
- Pouca abstração da estrutura física



Modelo Relacional

De Codd ao modelo relacional

Edgar Codd propôs o modelo relacional em 1970, revolucionando os sistemas de banco de dados ao organizar os dados de forma lógica e estruturada em tabelas.

Conceito central

- Dados organizados em tabelas, chamadas relações
- Cada linha representa uma tupla
- Cada coluna representa um atributo

Características-chave segundo Elmasri & Navathe

- **Simplicidade:** dados em tabelas simples e intuitivas
- **Independência de dados:** separação entre visão lógica e implementação física
- **Flexibilidade:** fácil de consultar e modificar
- **Integridade referencial:** relacionamentos por chaves estrangeiras

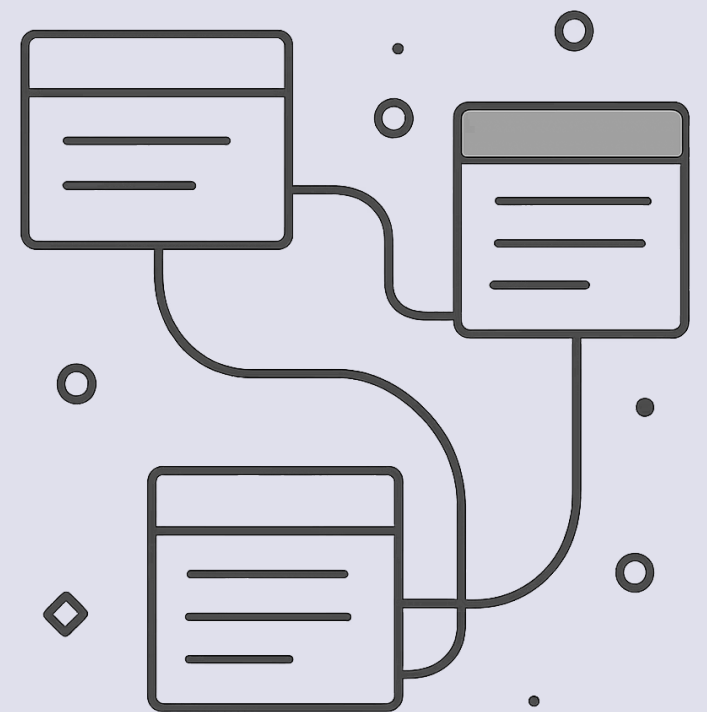
Vantagens

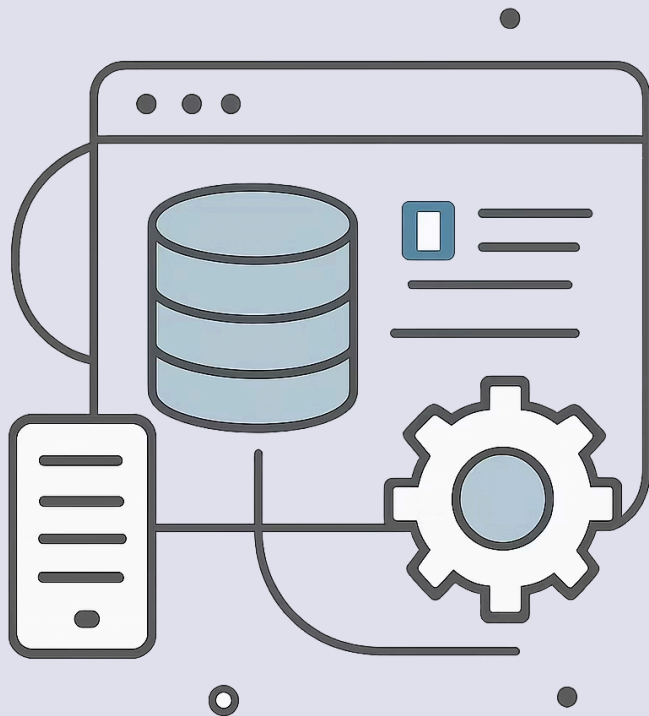
- Fácil de entender e usar
- Consultas poderosas através de SQL
- Suporta relacionamentos complexos de forma elegante
- Normalização garante consistência dos dados

Exemplo de estrutura

Em um banco universitário, **Alunos**, **Disciplinas** e **Professores** se relacionam por chaves estrangeiras, conectando registros entre tabelas.

Referência: Elmasri, R.; Navathe, S. B. *Sistemas de Banco de Dados*.





CAPÍTULO

Sistema Gerenciador de Banco de Dados

Entendendo o software que gerencia, armazena e recupera dados com segurança e eficiência.

O que é um SGBD?

É uma coleção de programas que permitem aos usuários definirem, construir e manipular uma base de dados para o uso das diversas aplicações.

Funções principais do SGBD

- Definir estrutura do banco de dados (DDL)
- Construir e popular o banco de dados
- Manipular dados (inserir, atualizar, deletar, consultar)
- Controlar acesso e segurança
- Garantir integridade e consistência dos dados

Componentes-chave

- Gerenciador de armazenamento
- Gerenciador de transações
- Gerenciador de consultas
- Gerenciador de segurança

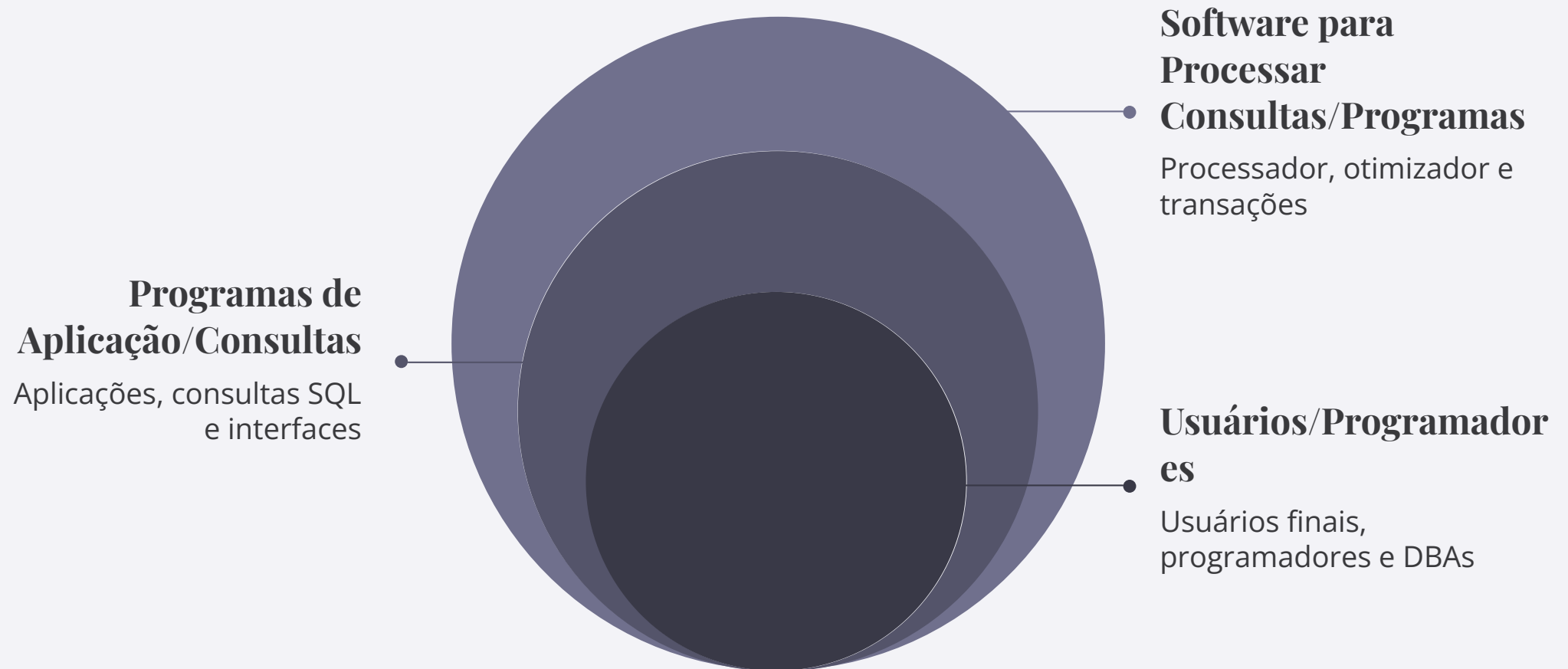
Exemplos de SGBDs

PostgreSQL, MySQL, Oracle, SQL Server, MongoDB



Arquitetura do SGBD

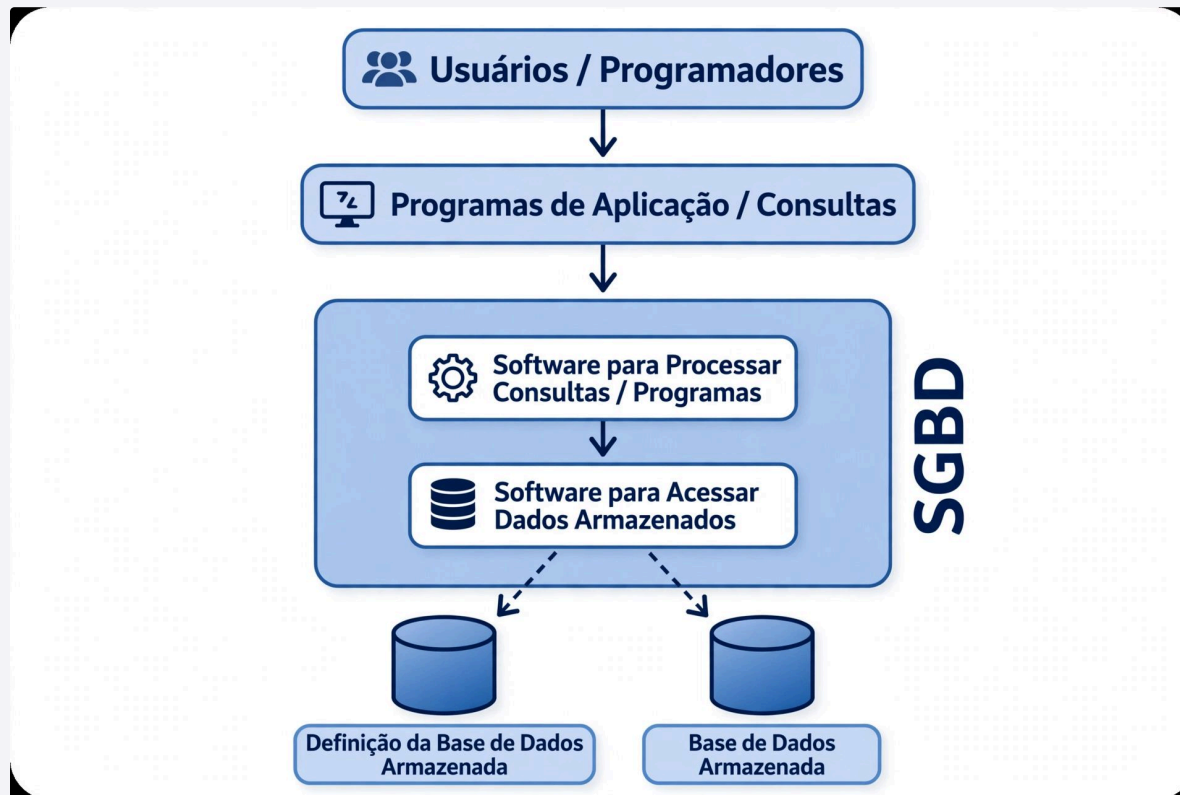
Visão em camadas baseada em Elmasri & Navathe, mostrando como usuários interagem com o SGBD por meio de interfaces, processamento de consultas e acesso aos dados armazenados.



A arquitetura separa a interação humana da manipulação física dos dados, criando camadas de abstração que facilitam uso, segurança, desempenho e manutenção.

Arquitetura do SGBD

Visão em camadas baseada em Elmasri & Navathe, mostrando como usuários interagem com o SGBD por meio de interfaces, processamento de consultas e acesso aos dados armazenados.



- A arquitetura separa a interação humana da manipulação física dos dados
- Criando camadas de abstração que facilitam uso, segurança, desempenho e manutenção.

Profissionais Envolvidos em um Banco de Dados

Administrador de Banco de Dados (DBA)

- Responsável por autorizar o acesso ao banco de dados
- Coordena e monitora seu uso
- Garante segurança e desempenho

Projetista do Banco de Dados

- Responsável pela identificação dos dados a serem armazenados
- Escolhe estruturas apropriadas para representar e armazenar dados
- Define o modelo conceitual e lógico

Usuários Finais

- Profissionais que precisam ter acesso ao banco de dados
- Consultam, modificam e removem dados
- Usam as aplicações desenvolvidas

Analistas de Sistemas e Programadores de Aplicação

- Analistas desenvolvem especificações das transações
- Atendem aos requisitos dos usuários
- Programadores implementam as especificações
- Devem estar familiarizados com todas as capacidades do SGBD

Surgimento de SGBDs

O Problema Original

- Programas gravavam seus dados em disco, segundo estruturas próprias
- Para acessá-los era necessário conhecer sua estrutura
- Cada programa tinha sua própria forma de armazenar dados
- Redundância de dados entre diferentes programas
- Dificuldade em compartilhar dados entre aplicações

Consequências

- Inconsistência de dados
- Desperdício de espaço em disco
- Dificuldade de manutenção
- Falta de segurança centralizada
- Problemas de concorrência

A Solução - SGBD

- Um software centralizado para gerenciar todos os dados
- Abstração da estrutura física dos dados
- Independência entre programas e dados
- Controle centralizado de acesso e segurança
- Eliminação de redundância através de normalização

Dependência de Dados em Programas Tradicionais

O Problema da Dependência de Dados

- Se vários programas compartilham seus dados, todos devem conhecer e manipular as mesmas estruturas
- Se algum programa precisar de alguma mudança na estrutura de dados, todos os programas terão que ser alterados
- Mesmo que a alteração ocorra em dados que ele não utiliza

Consequências da Dependência

- Acoplamento forte entre programas e dados
- Dificuldade de manutenção
- Risco de erros ao modificar estruturas
- Custo alto de manutenção
- Falta de flexibilidade

Exemplo Prático

- Se uma tabela de Clientes adiciona um novo campo (ex: CPF)
- Todos os programas que acessam Clientes precisam ser modificados
- Mesmo programas que não usam o campo CPF precisam ser alterados

A Solução: Independência de Dados

Colocar um sistema entre os dados e os programas, que converta o formato em que os dados estão gravados para o formato específico que cada programa precisa dos dados.

O Conceito

- Um intermediário (SGBD) entre os dados e os programas
- Converte o formato de armazenamento para o formato necessário
- Abstração da estrutura física dos dados

Como Funciona

- Programas solicitam dados ao SGBD
- SGBD recupera dados do armazenamento físico
- SGBD converte para o formato que o programa precisa
- Programa recebe dados no formato esperado

Benefícios

- Mudanças na estrutura física não afetam os programas
- Cada programa vê os dados no formato que precisa
- Múltiplos programas podem compartilhar os mesmos dados
- Manutenção centralizada no SGBD

Resultado

- Independência de Dados: programas independentes da estrutura física
- Flexibilidade: mudanças internas sem impacto nos programas
- Escalabilidade: fácil adicionar novos programas



CAPÍTULO 3

Propriedades de SGBDs

Características essenciais que garantem a qualidade, confiabilidade e integridade dos dados.

Propriedades de SGBDs – Parte 1: Integridade e Validade

Consistência de Dados

- O SGBD avalia os dados recebidos através de regras de integridade
- Garante que sempre estejam corretos
- Implementa validações automáticas

Validade

- Os dados são válidos quando pertencem ao domínio de valores possíveis naquele caso
- Exemplo: um campo de idade não pode ser negativo
- Exemplo: um email deve seguir o formato correto

Propriedades de SGBDs – Parte 2: Completeza e Consistência

Completeza

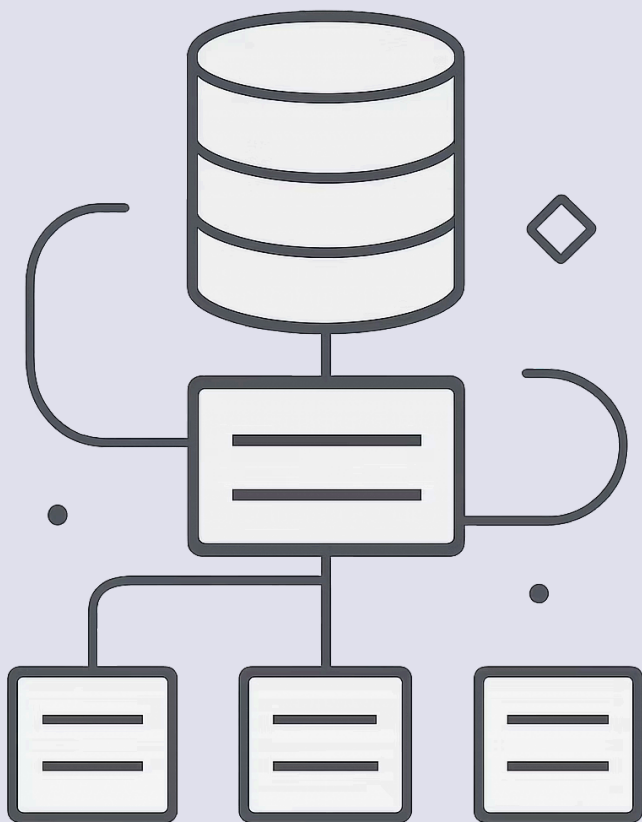
- Todos os dados que precisam ser conhecidos estão disponíveis
- Nenhum dado essencial está faltando
- O banco de dados contém todas as informações necessárias

Consistência

- Sempre que a mesma informação é gravada, mesmo que em locais diferentes, ela tem o mesmo valor
- Evita redundância e conflitos de dados
- Garante que múltiplas cópias dos mesmos dados permaneçam sincronizadas

Exemplos de SGBDs

ZIM	FoxPro	Dataflex
Unify	DB2	Paradox
Firebird	CA-OpenIngres	DBase
Jasmine	Informix	SQL-Server
O2	Oracle	Postgress
Orion	Sybase	Gupta
Access		



CAPÍTULO

Modelos de Banco de Dados

Três perspectivas para representar e organizar dados: do conceitual ao físico

Os Três Modelos de Banco de Dados



Modelo Conceitual

- Visão de alto nível
- Independente de qualquer SGBD
- Foca no "o quê" - quais dados precisam ser armazenados
- Usa diagramas Entidade-Relacionamento (ER)



Modelo Lógico

- Visão intermediária
- Dependente do tipo de SGBD (relacional, hierárquico, em rede)
- Foca no "como" - como os dados serão organizados
- Define tabelas, relacionamentos, atributos



Modelo Físico

- Visão de baixo nível
- Específico do SGBD e do hardware
- Foca na implementação - como os dados serão armazenados fisicamente
- Define índices, partições, alocação de espaço

Modelo Conceitual

1

Definição

- Representação de alto nível dos dados
- Independente de qualquer SGBD específico
- Foca no "o quê" - quais dados precisam ser armazenados
- Visão do negócio, não da tecnologia

2

Características

- Usa diagramas Entidade-Relacionamento (ER)
- Define entidades (objetos do mundo real)
- Define relacionamentos entre entidades
- Define atributos de cada entidade

3

Exemplo Prático - Sistema Universitário

- Entidades: Alunos, Disciplinas, Professores
- Relacionamentos: Aluno matricula-se em Disciplina, Professor leciona Disciplina
- Atributos: Aluno tem matrícula, nome, email; Disciplina tem código, nome, créditos

4

Vantagem

- Facilita comunicação entre analistas, usuários e desenvolvedores
- Independência de tecnologia
- Fácil de entender e modificar

Diagrama Entidade-Relacionamento (ER)

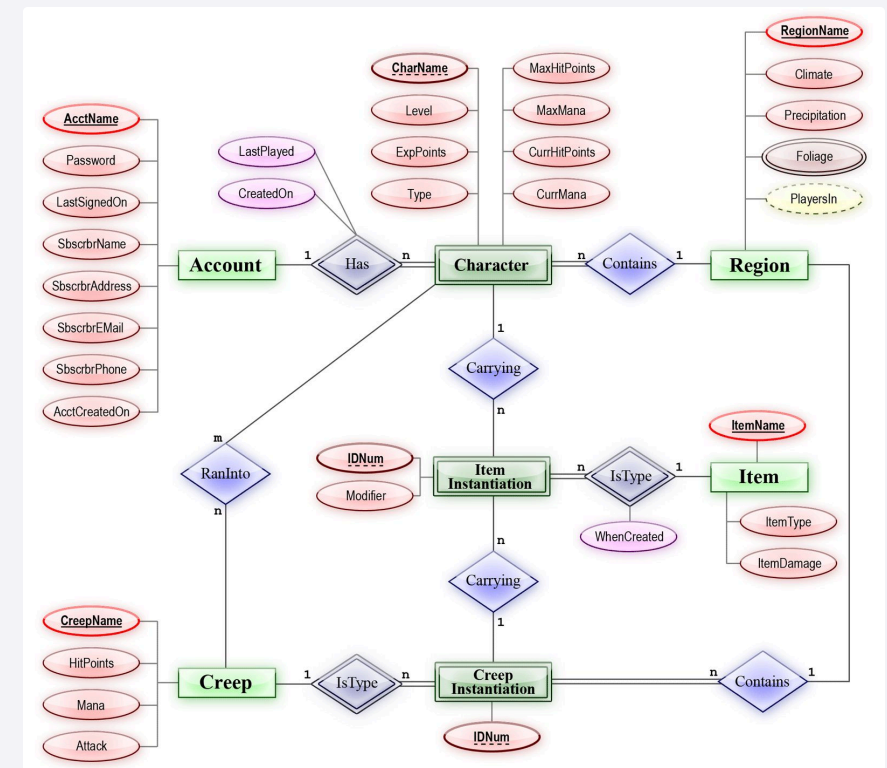
Ferramenta visual para modelagem conceitual.

Representa entidades, atributos e relacionamentos.

Base para o design do banco de dados.

Componentes principais

- Entidades: retângulos
- Atributos: elipses
- Relacionamentos: losangos
- Cardinalidade: notação 1:1, 1:N, N:N



Modelo Lógico

Definição

- Representação intermediária dos dados
- Dependente do tipo de SGBD escolhido
- Foca no "como" - como os dados serão organizados
- Traduz o modelo conceitual para estruturas do SGBD

Variações do Modelo Lógico (conforme o SGBD)

- Modelo Relacional: tabelas, linhas, colunas, chaves primárias e estrangeiras
- Modelo Hierárquico: estrutura em árvore, nós pai-filho
- Modelo em Rede: registros com múltiplos relacionamentos

Exemplo Prático - Modelo Relacional (Sistema Universitário)

- Tabela Alunos: matrícula (PK), nome, email
- Tabela Disciplinas: código (PK), nome, créditos
- Tabela Matrículas: matrícula (FK), código (FK), nota
- Define chaves primárias e estrangeiras

Características

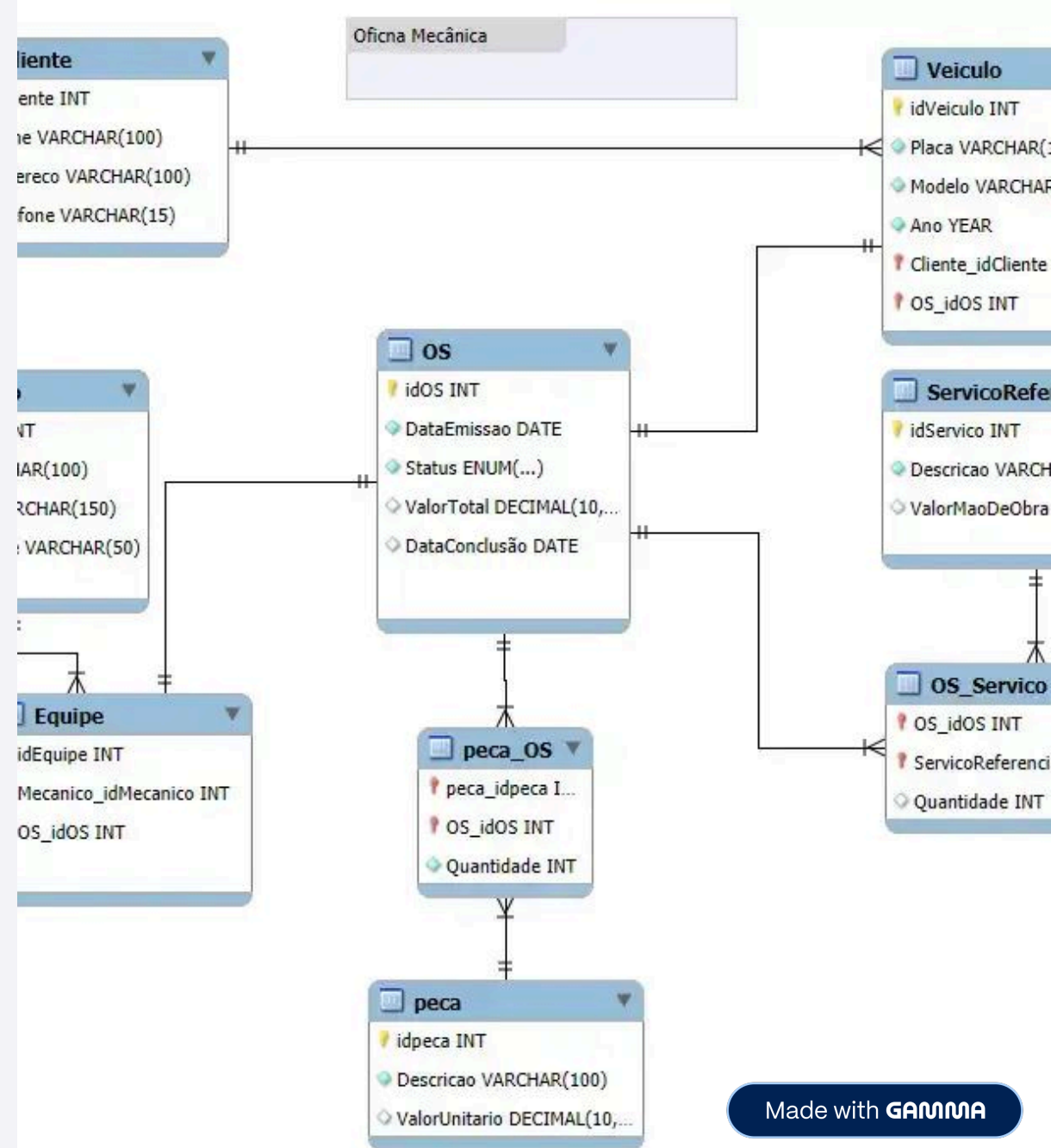
- Ainda independente do SGBD específico
- Fácil de validar e normalizar
- Serve como ponte entre conceitual e físico

Modelo Relacional

Organiza dados em tabelas (relações). Cada tabela contém linhas (tuplas) e colunas (atributos). Os relacionamentos entre tabelas ocorrem por meio de chaves primárias e estrangeiras.

Componentes principais

- Tabelas: estruturas de armazenamento
- Chaves Primárias (PK): identificam registros únicos
- Chaves Estrangeiras (FK): estabelecem relacionamentos
- Atributos: colunas da tabela



Modelo Físico

O Modelo Físico representa a implementação de baixo nível dos dados, focando em como eles serão armazenados e acessados no SGBD e hardware.

Definição

- Representação de baixo nível dos dados
- Específico do SGBD e do hardware utilizado
- Foca na implementação - como os dados serão armazenados fisicamente
- Detalha estruturas internas de armazenamento

Elementos do Modelo Físico

- Índices: estruturas para acelerar buscas
- Partições: divisão de tabelas grandes
- Alocação de espaço: como os dados ocupam o disco
- Compressão: técnicas para economizar espaço
- Replicação: cópias dos dados para redundância

Exemplo Prático - PostgreSQL (Sistema Universitário)

- Criar índice na coluna matrícula da tabela Alunos
- Particionar tabela Matrículas por ano
- Definir tablespace para armazenar dados
- Configurar backup e replicação

Características

- Específico do SGBD (PostgreSQL, Oracle, MySQL, etc.)
- Requer conhecimento técnico profundo
- Afeta desempenho e escalabilidade
- Pode ser modificado sem afetar modelos superiores

DDL e DML: Linguagens do Modelo Físico

DDL (Data Definition Language) e DML (Data Manipulation Language) são linguagens que operam no nível do modelo físico e permitem a implementação prática do banco de dados.

DDL (Data Definition Language)

- Responsável pela definição da estrutura física do banco de dados
- Define como os dados serão armazenados fisicamente
- Operações: CREATE (criar tabelas), ALTER (modificar estruturas), DROP (remover estruturas)
- Trabalha com a estrutura interna do SGBD

DML (Data Manipulation Language)

- Responsável pela manipulação dos dados armazenados
- Permite inserir, atualizar, deletar e consultar dados
- Operações: INSERT, UPDATE, DELETE, SELECT
- Trabalha com os dados já armazenados no banco

Por que são do Modelo Físico

- Operam diretamente no SGBD específico
- Dependem da implementação física do banco de dados
- Não são abstratas como o modelo conceitual ou lógico
- Executam operações reais de armazenamento e recuperação

Relação com os Modelos

- Modelo Conceitual: Define "o quê" armazenar (ER)
- Modelo Lógico: Define "como" organizar (tabelas, relacionamentos)
- Modelo Físico: Define "onde" e "como" implementar (DDL/DML)

Linguagens de Banco de Dados: DDL e DML

Linguagens para interagir com o banco de dados.

DDL

- CREATE: criar tabelas e estruturas
- ALTER: modificar estruturas existentes
- DROP: remover tabelas e estruturas

DML

- INSERT: adicionar novos registros
- UPDATE: modificar registros existentes
- DELETE: remover registros
- SELECT: consultar dados

Tabela de Vendedores

```
CREATE TABLE Vendedores (  
  VendedorID INT PRIMARY KEY IDENTITY,  
  Nome VARCHAR(255) NOT NULL,  
  Contato VARCHAR(255),  
  Telefone VARCHAR(50)
```

Tabela de Pedidos

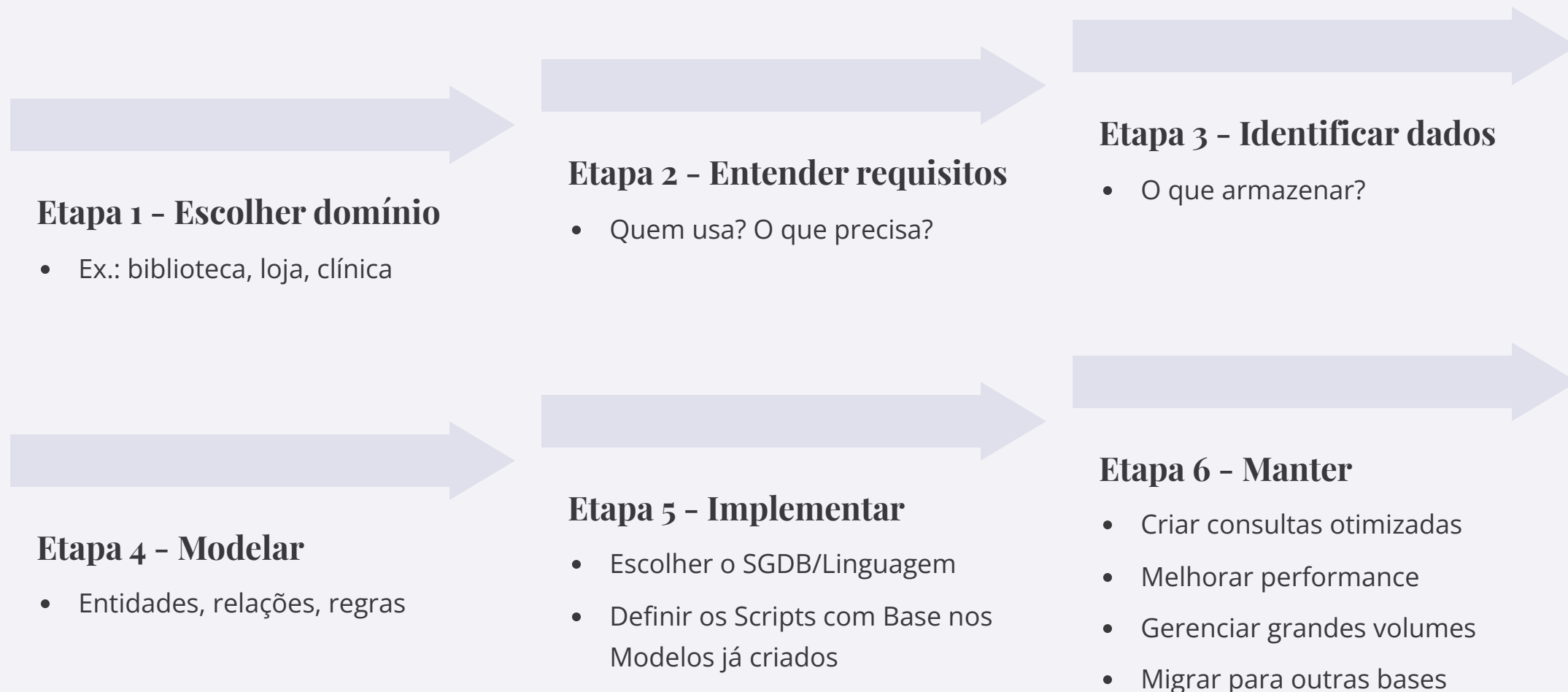
```
CREATE TABLE Pedidos (  
  PedidoID INT PRIMARY KEY IDENTITY,  
  DataPedido DATETIME NOT NULL,  
  ClienteID INT,  
  VendedorID INT,  
  StatusPedido VARCHAR(50),  
  FOREIGN KEY (ClienteID) REFERENCES Clientes(ClienteID),  
  FOREIGN KEY (VendedorID) REFERENCES Vendedores(VendedorID)
```

Tabela de Itens de Pedidos

```
CREATE TABLE ItensPedido (  
  ItemPedidoID INT PRIMARY KEY IDENTITY,  
  PedidoID INT,  
  ProdutoID INT
```

Ciclo de Vida - Projetar um Banco de Dados

O projeto começa agora — mesmo que a implementação venha depois.



IFPI CACAM - Banco de Dados I

Prof. Nairon Viana (nairon.viana@ifpi.edu.br)

Curso: Tecnólogo em ADS - 2026/02 (Módulo II)

